

Huffman Coding

Building an Optimal Tree from Frequencies

Unit III - Lecture 26

A tree built BOTTOM-UP from data, not top-down from a shape - and the reason common letters get short codes, rare ones get long codes.

The problem: fixed-length codes waste space

6 symbols need at least $\text{ceil}(\log_2 6) = 3$ bits each in a FIXED-length code, no matter how common or rare each one is.

But real text is skewed - some letters (e, a, t...) appear far more often than others (q, z, x...).

symbol	a	b	c	d	e	f
frequency	5	9	12	13	16	45

- f alone is 45 out of 100 characters - nearly half the text.
- Idea: give FREQUENT symbols SHORT codes and RARE symbols LONG codes - a VARIABLE-length code, chosen to minimize total bits.

Huffman's rule: merge the two smallest, every time

- 1 Put every symbol in a priority queue, ordered by frequency (smallest first).
- 2 Remove the TWO smallest. Create a new internal node whose frequency is their SUM, with the two removed nodes as its left and right children.
- 3 Insert that new node back into the queue. Repeat until only ONE node remains - the root.

This is the SAME priority-queue idea from Lecture 14 - insert freely, always extract the smallest. The GREEDY choice (always merge the two cheapest) is what makes the final tree optimal.

Trace: the first two merges (verified)

Queue (smallest first)	Action
a(5) b(9) c(12) d(13) e(16) f(45)	start
c(12) d(13) e(16) f(45) [14]	merge a(5)+b(9) -> [14]
e(16) f(45) [14] [25]	merge c(12)+d(13) -> [25]

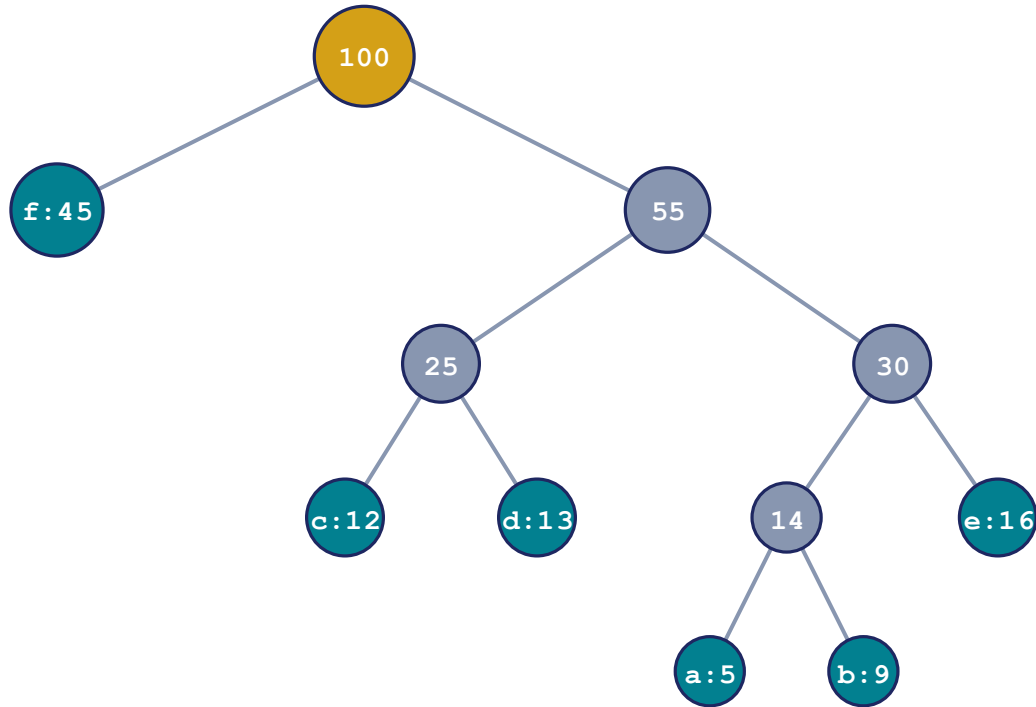
- a and b are the two SMALLEST (5 and 9) - they merge first, into a new internal node worth 14.
- That new [14] node re-enters the queue like any other item - it can be picked again later, exactly like any symbol.
- Next smallest pair: c(12) and d(13) - merge into [25].

Trace: merges continue until one node remains

Queue (smallest first)	Action
f(45) [14] [25]	e(16) waiting
f(45) [25] [30]	merge [14]+e(16) -> [30]
f(45) [55]	merge [25]+[30] -> [55]
[100]	merge f(45)+[55] -> [100] : the ROOT

The final [100] equals the total of all frequencies ($5+9+12+13+16+45=100$) - a useful sanity check: the root's frequency should always equal the sum of every leaf.

The tree those 5 merges actually built



0 = go left, 1 = go right (by convention)

Reading the codes: path from root to leaf

A symbol's Huffman code is the sequence of 0s and 1s on the path from the ROOT down to its leaf - 0 for every left turn, 1 for every right turn.

symbol	frequency	code	bits
f	45	0	1
e	16	111	3
d	13	101	3
c	12	100	3
b	9	1101	4
a	5	1100	4

Verified output. Notice: shortest code (1 bit) goes to the MOST frequent symbol; longest codes (4 bits) go to the two RAREST.

The payoff: how many bits does this actually save?

Fixed-length

6 symbols need 3 bits each.

100 characters x 3 bits
= 300 bits total.

Huffman (variable-length)

$45 \times 1 + 16 \times 3 + 13 \times 3 + 12 \times 3 + 9 \times 4 + 5 \times 4$
= 45+48+39+36+36+20
= 224 bits total.

224 bits instead of 300 - a real, verified saving of about 25%, purely from matching code length to how often each symbol actually appears.

Why 'always merge the two smallest' actually works

- The two least frequent symbols will always end up at the BOTTOM of any optimal tree - giving them the LONGEST codes wastes the fewest bits, since they occur the least.
- So it is always safe to lock them together as siblings FIRST, then treat the merged pair as one unit and repeat the same reasoning.
- This greedy choice at every step never has to be undone - a proof technique called an EXCHANGE ARGUMENT (any better tree can be rearranged into this one without increasing total cost).

Not every greedy algorithm is optimal - Huffman's is a genuine exception worth remembering, proven correct rather than just observed to work.

Decoding: why no code is a prefix of another

f=0, e=111, d=101, c=100, b=1101, a=1100 - no code here is the FIRST FEW BITS of another. This is called a PREFIX CODE, and it falls out automatically from the tree.

- Every symbol lives at a LEAF. A leaf can never be an ancestor of another leaf - so no code can be a prefix of a longer one.
- Decoding a bitstream: start at the root, follow 0/1 bits down the tree, and the moment you land on a leaf, you have found exactly one complete symbol - no separator characters needed.

ASK *Decode the bitstream 0 111 100 using the code table above, one bit-group at a time.*

A tree built from frequencies, not shape

- Repeatedly extract the two SMALLEST-frequency nodes from a priority queue, merge them into a new internal node (sum of frequencies), and reinsert - $n-1$ merges build the whole tree.
- A symbol's code is its root-to-leaf path (0=left, 1=right) - frequent symbols end up shallow (short codes), rare symbols end up deep (long codes).
- Verified saving on the running example: 224 bits (Huffman) vs 300 bits (fixed-length) - about 25% smaller.
- Every symbol lives at a leaf, so the code is automatically PREFIX-FREE - decoding needs no separators.

EXIT TICKET Symbols p(2), q(3), r(5), s(9). Build the Huffman tree by hand, showing each merge, and give every symbol's final code.